

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHẦN HIỆU TẠI TP.HCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



Kiểm thử Hộp trắng & Đo độ phủ JaCoCo

Buổi 3 / 9 — Môn: Kiểm thử Phần mềm

Nội dung buổi học

- 1 Kiểm thử hộp trắng: khái niệm, so sánh với hộp đen
- 2 Đồ thị luồng điều khiển (Control Flow Graph — CFG)
- 3 Các tiêu chí độ phủ: Câu lệnh (SC), Nhánh (BC), Đường đi (PC)
- 4 Độ phức tạp cyclomatic $V(G)$ (McCabe)
- 5 Kiểm thử vòng lặp (Loop Testing)
- 6 Kiểm thử luồng dữ liệu (Data Flow Testing)
- 7 Đo độ phủ tự động với JaCoCo trong DigiShield
- 8 Tổng kết & Hạn chế của hộp trắng

Vị trí trong đề cương môn học (Khung 9 buổi)

Buổi Chuyên đề

1–2	C1–C2: Tổng quan, quy trình (STLC)
3	C3: Kiểm thử hộp trắng & JaCoCo (hôm nay)
4	C4: Unit Test với JUnit 5 & Mockito
5	C4: Kiểm thử tích hợp (Testcontainers)
6	C5: Kiểm thử hộp đen (EP, BVA, Decision Table)
7	C5: Kiểm thử API (Postman / Newman)
8	C6: Kiểm thử Web UI (Selenium & POM)
9	C7: CI/CD Pipeline & Báo cáo BTL

Chương 3 — Hộp trắng

- 3.1. Đồ thị luồng điều khiển (CFG)
- 3.2. Phủ Statement, Branch, Path
- 3.3. Độ phức tạp $V(G)$ & Vòng lặp
- 3.4. Kiểm thử luồng dữ liệu & JaCoCo

Mục tiêu hôm nay

Đo độ phủ code tự động bằng **JaCoCo** trên repo DigiShield, chuẩn bị viết unit test.

Ôn tập buổi 2 — vì sao cần hộp trắng?

Buổi 2 đã học

- Quy trình kiểm thử (STLC), V-model
- Các mức kiểm thử: unit, integration, system, acceptance
- Testing pyramid: unit test là tầng đáy
- Đăng ký Bài tập lớn theo nhóm

Câu hỏi mở đầu

Nếu chỉ nhìn từ bên ngoài (nhập input, xem output), ta **không biết**:

- Nhánh code nào **chưa từng** được chạy?
- Có **dead code** không?
- Bộ test hiện tại “tốt” đến mức nào?

⇒ Cần nhìn **vào bên trong** mã nguồn: kiểm thử hộp trắng.

Kiểm thử hộp trắng là gì?

Định nghĩa

Kiểm thử hộp trắng (white-box testing) thiết kế test case **dựa trên cấu trúc bên trong** của phần mềm: mã nguồn, luồng điều khiển, luồng dữ liệu.^[1,2]

Tên gọi khác:

- Structural testing (kiểm thử cấu trúc)
- Glass-box / clear-box testing
- Code-based testing

Mục tiêu:

- Mọi câu lệnh được thực thi ít nhất một lần
- Mọi nhánh true/false đều được đi qua
- Phát hiện dead code, nhánh cài đặt sai
- **Đo được** chất lượng bộ test bằng độ phủ (coverage)

Hộp trắng vs Hộp đen

	Hộp trắng	Hộp đen
Cơ sở thiết kế test	Mã nguồn, CFG, luồng dữ liệu	Đặc tả, yêu cầu, giao diện
Cần biết code?	Có (đọc và hiểu code)	Không
Ai thực hiện	Developer, tester biết lập trình	Tester, BA, người dùng
Mức áp dụng chủ yếu	Unit test, integration test	System test, acceptance test
Phát hiện tốt	Nhánh sai, dead code, lỗi logic cài đặt	Thiếu chức năng, sai so với yêu cầu
Thước đo	Độ phủ Câu lệnh (SC) / Nhánh (BC) / Đường đi (PC)	Độ phủ yêu cầu, lớp tương đương

Hai kỹ thuật bổ sung cho nhau: hộp trắng trả lời “code **đã viết** có được test đủ chưa?”; hộp đen trả lời “phần mềm có làm **đúng yêu cầu** không?”.

Khi nào dùng kiểm thử hộp trắng?

Tình huống điển hình:

- Viết **unit test** cho service, hàm tiện ích (buổi 5–6 sẽ cài đặt bằng JUnit/Mockito)
- Code review: chỉ ra nhánh chưa được test
- Nâng độ phủ theo yêu cầu dự án (DigiShield đang ép line coverage $\geq$ 50%)
- Kiểm thử logic nghiệp vụ nhiều rẽ nhánh: đánh giá rủi ro, quyết định **tạm dừng / cảnh báo / cho qua** một giao dịch

Trên DigiShield hôm nay

Ta sẽ phân tích 2 method **thật**:

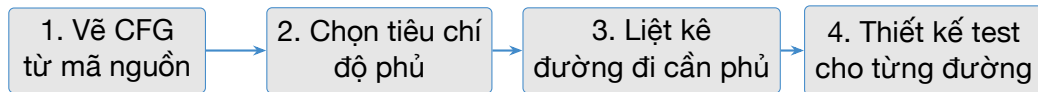
- `ReportingServiceImpl.ageLabel()` — nhãn tuổi báo cáo phishing
- `InterceptionServiceImpl.evaluate()` — quyết định tạm dừng, cảnh báo hoặc cho qua giao dịch

và đối chiếu với **unit test thật** trong repo.

Kiểm thử luồng điều khiển — quy trình 4 bước

Control flow testing^[2]

Kỹ thuật hộp trắng chọn test case dựa trên các **đường thực thi** (execution path) qua mã nguồn.



- Bước 2: Câu lệnh (SC), Nhánh (BC), Đường đi (PC)...
- Bước 4: tìm **input** khiến chương trình đi đúng đường đã chọn.

Control Flow Graph (CFG — Đồ thị luồng điều khiển)

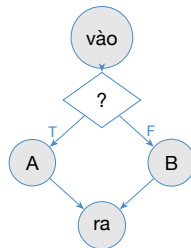
Định nghĩa

CFG là đồ thị có hướng $G = (N, E)$:

- N : tập **node** — mỗi node là một khối câu lệnh tuần tự (basic block)
- E : tập **cạnh có hướng** — mỗi cạnh là một bước chuyển điều khiển có thể xảy ra

Các loại node:

- **Entry node**: điểm vào duy nhất của method
- **Exit node**: điểm kết thúc
- **Node quyết định** (decision): có ≥ 2 cạnh ra — sinh từ if, while, for, switch
- **Node hợp** (junction): có ≥ 2 cạnh vào

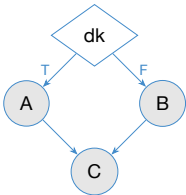


"?" = node quyết định; "ra" = node hợp.

Vẽ CFG từ cấu trúc rẽ nhánh

if có else

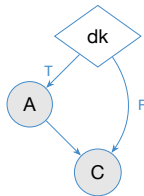
```
if (dk) { A; }  
else    { B; }  
C;
```



2 nhánh: T và F.

if KHÔNG có else

```
if (dk) { A; }  
C;
```

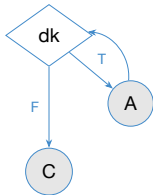


Nhánh F là cạnh “đi tắt” — **rất dễ quên khi test!**

Vẽ CFG từ vòng lặp và switch

while / for

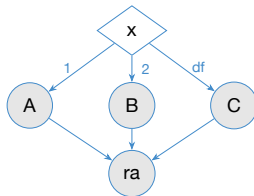
```
while (dk) { A; }  
C;
```



Cạnh quay lui (back edge) tạo **chu trình** $\Rightarrow$ số đường đi bùng nổ theo số lần lặp.

switch / if-else if-else

```
switch (x) {  
  case 1: A; break;  
  case 2: B; break;  
  default: C;  
}
```



Node quyết định có thể có **nhiều hơn 2** cạnh ra.

Quy ước vẽ CFG trong môn học

- 1 Gộp các câu lệnh tuần tự liên tiếp thành **một node** (basic block) — đồ thị gọn, không mất thông tin.
- 2 Mỗi điều kiện rẽ nhánh vẽ bằng **hình thoi**, ghi rõ cạnh **T** (true) và **F** (false).
- 3 Method có nhiều return: nối tất cả điểm return về **một exit node chung** — để đường đi dễ nhìn hơn.
- 4 Đánh **số thứ tự** node để mô tả đường đi, ví dụ: $1 \rightarrow 2(T) \rightarrow 3 \rightarrow 7$.

Lỗi hay gặp của sinh viên

Quên nhánh F của `if` không có `else`; quên exit node chung khi method có nhiều return; vẽ mỗi dòng code một node làm đồ thị rối.

Ví dụ 1: bối cảnh nghiệp vụ ageLabel()

Hàm này giải quyết bài toán gì?

Trong màn hình quản trị báo cáo lừa đảo, hệ thống cần hiển thị thời điểm người dùng gửi báo cáo theo dạng **ngắn, dễ đọc**: "30m", "2h", "3d"
(m = phút/minutes, h = giờ, d = ngày.)

Input

- reportedAt: thời điểm báo cáo được gửi
- now: thời điểm hiện tại để so sánh

Output mong đợi

- Dưới 60 phút → "30m" = 30 phút
- Dưới 24 giờ → "2h" = 2 giờ
- Từ 1 ngày trở lên → "3d" = 3 ngày

Vì sao chọn làm ví dụ CFG đầu tiên?

Hàm ngắn, không gọi DB, không phụ thuộc network; mỗi nhánh trả về một nhãn khác nhau nên rất dễ thấy quan hệ giữa **code** → **CFG** → **test case**.

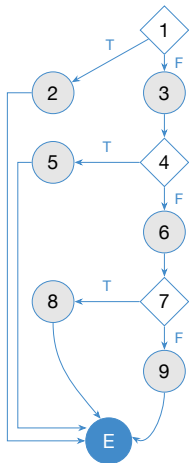
Ví dụ 1: ageLabel() — code thật

ReportingServiceImpl.ageLabel() — nhãn tuổi báo cáo (“2m”, “3h”, “1d”)

```
1 private String ageLabel(Instant reportedAt, Instant now) {
2     if (reportedAt == null) {                // node 1
3         return null;                        // node 2
4     }
5     // node 3: gop 2 cau lenh tuan tu (tinh d, mins thanh 1 basic block)
6     Duration d = Duration.between(reportedAt, now);
7     long mins = Math.max(0, d.toMinutes());
8     if (mins < 60) {                        // node 4
9         return mins + "m";                 // node 5 - "N phut"
10    }
11    long hours = d.toHours();                // node 6 (tinh hours)
12    if (hours < 24) {                       // node 7
13        return hours + "h";                // node 8 - "N gio"
14    }
15    return d.toDays() + "d";               // node 9 - "N ngay"
16 }
```

Hàm **thuần** (pure): không phụ thuộc DB $\Rightarrow$ dễ test. 3 điểm quyết định, 4 kết quả loại trừ nhau.

CFG của ageLabel()



Node	Nội dung
1	if (reportedAt == null)
2	return null
3	tính d, mins
4	if (mins < 60)
5	return mins + "m"
6	tính hours
7	if (hours < 24)
8	return hours + "h"
9	return days + "d"
E	exit chung (quy ước)

Đếm: $N = 10$ node, $E = 12$ cạnh, **3 node quyết định** (1, 4, 7), **4 đường đi** từ vào đến ra.

ageLabel() — 4 đường đi và test data

Path	Đường đi	Test data (reportedAt so với now)	Expected
P1	$1(T) \rightarrow 2 \rightarrow E$	reportedAt = null	null
P2	$1(F) \rightarrow 3 \rightarrow 4(T) \rightarrow 5 \rightarrow E$	30 phút trước	"30m"
P3	$1(F) \rightarrow 3 \rightarrow 4(F) \rightarrow 6 \rightarrow 7(T) \rightarrow 8 \rightarrow E$	2 giờ trước	"2h"
P4	$1(F) \rightarrow 3 \rightarrow 4(F) \rightarrow 6 \rightarrow 7(F) \rightarrow 9 \rightarrow E$	3 ngày trước	"3d"

Nhận xét

- 4 test case $\Rightarrow$ đi hết mọi node, mọi cạnh, mọi đường đi.
- Giá trị **biên** 59/60 phút, 23/24 giờ chưa nằm trong bảng — đó là việc của BVA (buổi 8). Hộp trắng và hộp đen bổ sung nhau!

Ví dụ 2: bối cảnh nghiệp vụ evaluate()

Module interception — xử lý giao dịch đáng ngờ

Nạn nhân lừa đảo qua điện thoại thường chuyển tiền **trong khi vẫn đang nghe kẻ lừa đảo thao túng**. DigiShield đánh giá mỗi giao dịch theo 3 tín hiệu:

- onCall — người dùng đang trong cuộc gọi?
- newPayee — chuyển cho người thụ hưởng mới?
- watchlistHit — tài khoản đích nằm trong danh sách theo dõi?

Luật quyết định (code thật)

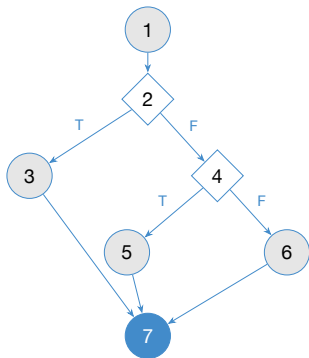
- Cả 3 tín hiệu cùng bật ⇒ **PAUSE** (tạm dừng giao dịch, hiện cảnh báo giáo dục)
- Chỉ trúng watchlist ⇒ **WARN** (cảnh báo nhưng cho phép)
- Còn lại ⇒ **ALLOW** (cho qua)

API: POST /api/v1/interventions/evaluate

evaluate() — code thật (rút gọn)

```
1 // import static ...domain.InterventionSignal.*;
2 public InterventionDecision evaluate(EvaluateRequest req) {
3     List<InterventionSignal> signals = new ArrayList<>();
4     if (req.onCall()) signals.add(ON_CALL); // S1
5     if (req.newPayee()) signals.add(NEW_PAYEE); // S2
6     boolean watchlistHit = watchRepository
7         .findFirstByTenantIdAndValueOrderByAddedAtDesc(
8             tenantId, req.destAccount())
9         .isPresent();
10    if (watchlistHit) signals.add(WATCHLIST_HIT); // S3
11    Decision decision;
12    if (req.onCall() && req.newPayee() && watchlistHit) {
13        decision = Decision.PAUSE; // D1
14    } else if (watchlistHit) {
15        decision = Decision.WARN; // D2
16    } else {
17        decision = Decision.ALLOW;
18    }
19    eventRepository.save(event); // lưu: name() -> CHU HOA
20    return new InterventionDecision( // tra ve: chu thuong
21        decision.name().toLowerCase(Locale.ROOT),
22        signals.stream()
23            .map(InterventionSignal::wireName).toList(),
24        message);
25 }
```

CFG của evaluate() – phần quyết định chính



Lưu ý: đang gộp S1–S3 để học 3 nhánh PAUSE / WARN / ALLOW trước; slide sau mở S1–S3 $\Rightarrow$ số nhánh và $V(G)$ tăng.

Node	Nội dung
1	Gom signals, tra watchlist
2	onCall && newPayee && hit?
3	decision = PAUSE
4	watchlistHit?
5	decision = WARN
6	decision = ALLOW
7	save + return (exit)

Đếm: $N = 7$, $E = 8$, 2 node quyết định (2, 4).

3 đường đi:

- 1 $1 \rightarrow 2(T) \rightarrow 3 \rightarrow 7$ — PAUSE
- 2 $1 \rightarrow 2(F) \rightarrow 4(T) \rightarrow 5 \rightarrow 7$ — WARN
- 3 $1 \rightarrow 2(F) \rightarrow 4(F) \rightarrow 6 \rightarrow 7$ — ALLOW

Độ phủ câu lệnh (Statement Coverage — SC)

Định nghĩa^[1,3]

$$SC = \frac{\text{số câu lệnh đã được thực thi bởi bộ test}}{\text{tổng số câu lệnh thực thi được}} \times 100\%$$

Đạt 100% khi **mỗi câu lệnh** (mỗi node của CFG) được thực thi ít nhất một lần.

- Là tiêu chí **yếu nhất** — mức tối thiểu mà mọi bộ unit test nên đạt cho code nghiệp vụ.
- Phát hiện được: dead code, câu lệnh ném exception chưa từng chạy.
- Công cụ đo tự động: JaCoCo (xem phần cuối buổi 3) — hôm nay ta **tính tay** trên CFG để hiểu bản chất.

Tính tay SC trên CFG của evaluate()

Test	Node đi qua	Node đã phủ	SC
T1: PAUSE (onCall+newPayee+watchlist)	1, 2, 3, 7	4/7	57%
+ T2: WARN (watchlist)	thêm 4, 5	6/7	86%
+ T3: ALLOW (không watchlist)	thêm 6	7/7	100%

Kết luận

Cần tối thiểu **3 test case** để đạt 100% SC — vì ba node 3, 5, 6 nằm trên ba nhánh **loại trừ nhau**, không test nào đi qua hai node cùng lúc.

Câu hỏi nhanh: với `ageLabel()`, cần bao nhiêu test để đạt 100% SC? (Trả lời: 4 — bốn return loại trừ nhau.)

Độ phủ nhánh (Branch/Decision Coverage — BC)

Nguyên tắc: Mọi ngã rẽ đều phải test cả 2 hướng

Tại mỗi câu lệnh rẽ nhánh (`if`, `while`, `for`), luồng code tách đôi thành **True** và **False** (kể cả khi code không viết chữ `else`).

$$BC = \frac{\text{Số nhánh True / False đã test}}{\text{Tổng số nhánh True / False}} \times 100\%$$

So sánh với Statement Coverage (SC)

- **100% BC $\Rightarrow$ 100% SC:** Đi hết mọi ngã rẽ thì chắc chắn ghé qua mọi dòng code.
- **Ngược lại KHÔNG đúng:** 100% SC vẫn có thể bỏ sót nhánh False của câu `if` không có `else`.

Quy tắc đếm nhanh

- 1 câu `if` $\rightarrow$ **2 nhánh** (T / F)
- 3 câu `if` $\rightarrow$ **6 nhánh**
- **BTL yêu cầu:** $BC > 90\%$
- (SonarQube chuẩn: 80%)

Tính tay BC: T1–T4 là gì?

Quy ước đọc

Mỗi test là một bộ 3 giá trị theo thứ tự:
(onCall, newPayee, watchlistHit).

T = đúng, **F** = sai.

Nhóm có watchlist

- T1=(T,T,T) $\Rightarrow$ **PAUSE**
đủ 3 dấu hiệu nguy hiểm.
- T2=(F,F,T) $\Rightarrow$ **WARN**
chỉ trúng watchlist.

Nhóm không có watchlist

- T3=(T,T,F) $\Rightarrow$ **ALLOW**
không trúng watchlist.
- T4=(F,F,F) $\Rightarrow$ **ALLOW**
không có tín hiệu rủi ro.

Chốt: T1–T4 là **4 bộ dữ liệu đầu vào**; slide sau dùng chúng để đánh dấu nhánh True/False nào đã được test.

Tính tay BC: kiểm 10 nhánh T/F

Mục tiêu của BC

Với 5 node quyết định, mỗi node phải có ít nhất một test đi qua nhánh **True** và ít nhất một test đi qua nhánh **False**.

Node	Câu hỏi	Nhánh T	Nhánh F
S1	Đang gọi điện?	T1, T3	T2, T4
S2	Người nhận mới?	T1, T3	T2, T4
S3	Trúng watchlist?	T1, T2	T3, T4
D1	Có đủ 3 tín hiệu để PAUSE?	T1	T2, T3, T4
D2	Nếu không PAUSE, có WARN không?	T2	T3, T4

Kết luận

Tất cả 10 ô T/F đều có ít nhất một test phủ $\Rightarrow$ **BC = 10/10 = 100%**.

Ghi nhớ: 3 test PAUSE / WARN / ALLOW đủ cho **phần quyết định chính**; thêm T4 để phủ rõ các nhánh S1–S3.

Vì sao 100% SC vẫn chưa đủ? (Cái bẫy nhánh ẩn)

Code: if không có else

```
String verdict = "flag";  
if (reasons.size() >= 2) {  
    verdict = "block";  
}
```

Chạy 1 test reasons.size() = 2:

- Cả 3 dòng đều chạy ⇒ **SC = 100%** (báo xanh!)
- Nhưng nhánh **False** (khi size < 2) **chưa từng được test!**

Nguy cơ: Bug lọt lưới!

Nếu dev viết nhầm: size() >= 1

- Test size=2: vẫn ra "block" → **Vẫn PASS!**
- Bộ test SC 100% hoàn toàn bị lừa!
- Chỉ khi test nhánh False (size=1) mới lộ kết quả sai.

⇒ **Độ phủ nhánh (BC) ép test cả False**
→ **Tóm được bug!**

Khẩu quyết ghi nhớ

Câu if không có else luôn ẩn giấu một **nhánh False đi tắt**. Muốn không sót lỗi, bắt buộc phải dùng **Độ phủ nhánh (BC)**!

Độ phủ đường đi (Path Coverage — PC)

Định nghĩa

Mỗi **đường đi** (path) khả thi từ entry đến exit của CFG phải được thực thi ít nhất một lần.

Trên ví dụ hôm nay

- `evaluate()` phân quyết định chính: **3** đường đi PAUSE / WARN / ALLOW.
- `ageLabel()`: **4** đường đi.

Ghi nhớ

PC mạnh hơn BC, nhưng dễ tăng rất nhanh khi code có nhiều `if` hoặc vòng lặp. Vì vậy thực tế thường chọn **một tập đường đi chính** để test.

Độ phức tạp cyclomatic $V(G)$ — McCabe

Bản chất: $V(G)$ là gì?

$V(G)$ là thước đo **độ phức tạp logic** của một method.

⇒ Trả lời ngay câu hỏi: **“Cần viết bao nhiêu test case?”**

Ý nghĩa với Tester:

- **Đường đi cơ sở (Basis Paths):** Thay vì test hàng trăm tổ hợp, chỉ cần chọn đúng $V(G)$ kịch bản chạy độc lập để vét sạch mọi ngã rẽ.
- **Số test case:** Cần đúng $V(G)$ test để phủ 100% đường đi cơ sở, và $V(G)$ là **mốc tối đa** để phủ 100% nhánh.

Thang đo chất lượng code

- **1 – 10:** Code tốt, gọn, dễ test.
- **11 – 20:** Khá rối, bắt đầu rủi ro.
- **> 20:** Code rối như mớ bòng bong, rất khó test!

Quy tắc vàng: Nếu $V(G) > 10 \Rightarrow$ bắt buộc phải tách nhỏ method (refactor).

Tính $V(G)$ bằng cách đếm decision

Công thức dùng nhanh khi nhìn code

$$V(G) = D + 1$$

D là số điểm rẽ nhánh: `if`, `else if`, `for`, `while`, `case`.

Cách nhớ

- Chỉ đếm các điểm hỏi **có / không**.
- `else` không tính thêm vì là nhánh `False` có sẵn.
- Mỗi decision thêm một ngã rẽ; cộng thêm 1 cho đường đi ban đầu.

Ví dụ tính $V(G)$

ageLabel()

- 3 decision
- $V(G) = D + 1 = 3 + 1 = 4$
- Basis path = 4 đường P1-P4 (slide trước) $\Rightarrow$ tối thiểu **4 test case**

evaluate() — phần quyết định chính

- 2 decision: PAUSE? WARN?
- $V(G) = D + 1 = 2 + 1 = 3$
- Basis path = 3 đường PAUSE / WARN / ALLOW $\Rightarrow$ tối thiểu **3 test case**

Chốt cách hiểu

Chọn **một phạm vi CFG** rồi tính theo phạm vi đó. Ở đây evaluate() chỉ tính phần quyết định chính.

Đối chiếu với unit test THẬT trong repo

```
modules/interception/src/test/.../InterceptionServiceImplTest.java
```

Các test method khớp đúng bảng T1–T4 ta vừa thiết kế:

- 1 `evaluateWhenOnCallAndNewPayeeAndWatchlistHitPauses...` → T1
- 2 `evaluateWhenWatchlistHitButNotOnCallNorNewPayeeWarns` → T2
- 3 `evaluateWhenOnCallAndNewPayeeButNoWatchlistHit...Allows` → T3
- 4 `evaluateWhenNoSignalsAllowsAndPersistsEmptySignals` → T4
- 5 `checkAccountDelegatesToRepositoryScopedByTenant` (ngoài phần chính của `evaluate`)

Chạy thử ngay trên máy (không cần Docker/DB)

```
./gradlew :modules:interception:test --tests "InterceptionServiceImplTest"
```

Toàn bộ test PASS trong 1–2 giây — chứng minh 100% độ phủ nhánh (BC) cho hàm `evaluate()`.

Test thật T1 & T4 trong repo DigiShield

T1: Nhánh PAUSE (T, T, T)

```
1 @Test
2 void evaluateWhenOnCallAndNewPayee...Pauses() {
3     when(watchRepository
4         .findFirstByTenantIdAndValueOrderByAddedAtDesc(
5             TENANT_ID, DEST_ACCOUNT))
6         .thenReturn(Optional.of(watchHit));
7     EvaluateRequest req = new EvaluateRequest(
8         userId, new BigDecimal("5000000"),
9         DEST_ACCOUNT, true, true); // onCall=T, newPayee=T
10
11     InterventionDecision d = service.evaluate(req);
12
13     assertThat(d.decision()).isEqualTo("pause");
14     assertThat(d.signals()).containsExactly(
15         "on_call", "new_payee", "watchlist_hit");
16 }
```

T4: Nhánh ALLOW (F, F, F)

```
1 @Test
2 void evaluateWhenNoSignalsAllowsAndPersistsEmptySignals() {
3     when(watchRepository
4         .findFirstByTenantIdAndValueOrderByAddedAtDesc(
5             TENANT_ID, DEST_ACCOUNT))
6         .thenReturn(Optional.empty());
7     EvaluateRequest req = new EvaluateRequest(
8         userId, new BigDecimal("10"),
9         DEST_ACCOUNT, false, false); // Tất cả false
10
11     InterventionDecision d = service.evaluate(req);
12
13     assertThat(d.decision()).isEqualTo("allow");
14     assertThat(d.signals()).isEmpty();
15     verify(eventRepository).save(eventCaptor.capture());
16 }
```

Khớp lý thuyết 100%

T1 kiểm tra PAUSE khi onCall + newPayee + watchlistHit đều True; T4 kiểm tra ALLOW khi mọi tín hiệu đều False.

Tự chạy bộ test thật

Lệnh chạy (không cần Docker)

```
cd digishield  
./gradlew :modules:interception:test --tests "InterceptionServiceImplTest"
```

- Kỳ vọng: **Toàn bộ test PASS** trong 1–2 giây — unit test thuần Mockito, không cần database.
- Thử “phá”: sửa điều kiện trong `evaluate()` (ví dụ bỏ `&& watchlistHit`) rồi chạy lại — test nào FAIL? Nhánh nào đã bắt được lỗi?
- Phần sau của buổi học sẽ dùng **JaCoCo** xác nhận bằng máy:
`./gradlew testCodeCoverageReport`.

Vòng lặp trong luồng điều khiển — Loop Testing

Vì sao không thể test mọi số lần lặp?

Vòng lặp tạo ra vô số đường đi (0, 1, 2, ... lần lặp). Kỹ thuật **Loop Testing** chọn ra **các mốc giá trị biên** để kiểm thử.^[3]

Quy ước n và m

- n = **Số lần lặp TỐI ĐA**:
Trần tối đa theo nghiệp vụ (VD: mảng max $n = 10$ phần tử, OTP thử max 5 lần).
- m = **Số lần lặp ĐIỂN HÌNH**:
Giá trị ở khoảng giữa $2 < m < n$ (VD: với $n = 10$, chọn $m = 4$).

Ý nghĩa các mốc kiểm thử

0 lần	Rỗng (phủ nhánh False)
1 lần	Biên min (khởi tạo đúng?)
2 lần	Bước lặp $i++$ chạy đúng?
m lần	Kịch bản thông thường
$n - 1, n$	Chạm trần (lỗi off-by-one)
$n + 1$	Vượt trần (chặn tràn mảng)

Ví dụ: Vòng lặp đếm từ khóa lừa đảo trong DigiShield

Hàm `classify()` trong `StubAiClient.java` (Module AI)

Kiểm tra một đoạn tin nhắn xem chứa bao nhiêu từ khóa lừa đảo trong bộ 10 từ khóa ("password", "otp", "urgent"...):

```
1 // signals = danh sách 10 từ khóa lừa đảo (n = 10)
2 long hits = 0;
3 for (String keyword : signals) {
4     if (payload.contains(keyword)) {
5         hits++; // Đếm số từ khóa xuất hiện trong tin nhắn
6     }
7 }
8 if (hits > 0) return "threat"; // Có dấu hiệu lừa đảo
9 return "clean";               // Tin nhắn an toàn
```

Câu hỏi đặt ra cho Tester

Danh sách có $n = 10$ từ khóa. Ta nên test những kịch bản nào để vừa vét sạch lỗi vòng lặp và câu lệnh cộng dồn `hits++`, vừa không tốn công vô ích?

Thiết kế ca kiểm thử theo nguyên tắc Loop Testing

5 kịch bản kiểm thử theo mốc lặp chuẩn

Mốc lặp	Dữ liệu tin nhắn	Ý nghĩa kiểm thử
0 lần	"Xin chào ngay moi"	0 hit → Trả về "clean"
1 lần	"Nhập password ngay"	1 hit → Biên min phát hiện
2 lần	"Password va otp"	2 hit → Kiểm tra bước hits++
$m = 5$	Chứa 5 từ khóa	Trần điểm tin cậy (1.0)
$n = 10$	Chứa toàn bộ 10 từ	Trần vòng lặp (khớp 10/10)

Bài học rút ra

- **Không test tràn lan** cả 10 trường hợp!
- Chọn đúng 5 mốc: **0** (rỗng), **1** (min), **2** (bước lặp), m (điểm hình/trần điểm), và n (trần lặp).
- Phân biệt rõ: $m = 5$ là trần điểm số, còn $n = 10$ là trần cấu trúc danh sách.

Kiểm thử luồng điều khiển KHÔNG phát hiện được gì?

Hạn chế

- **Code thiếu:** yêu cầu chưa được cài đặt thì không có nhánh nào để phủ.
- **Lỗi đặc tả:** code chạy đúng như dev viết, nhưng dev hiểu sai yêu cầu
- **Lỗi dữ liệu / biên:** 100% BC vẫn lọt lỗi $<$ vs $\leq$ tại biên

Cách khắc phục

- Kết hợp **hộp đen** để soi từ phía yêu cầu.
- Mỗi yêu cầu nên có ít nhất 1 test case.
- Review chéo đặc tả và code.

Ghi nhớ

Độ phủ đo “bộ test đã chạm bao nhiêu **code**”, không đo “phần mềm đúng bao nhiêu **yêu cầu**”.

Data Flow Testing: lần theo giá trị của biến

Câu hỏi chính

Control Flow hỏi: **test đã đi qua nhánh nào?**

Data Flow hỏi: **giá trị tạo ra ở một chỗ có được dùng đúng ở chỗ sau không?**

Cách đọc đơn giản

- 1 Tìm nơi biến được **gán**.
- 2 Tìm nơi biến được **đọc lại**.
- 3 Viết assert ở nơi có thể quan sát kết quả.

Lỗi thường gặp

- Gán đúng nhưng không dùng.
- Dùng nhầm biến cũ.
- Response đúng nhưng DB event lưu sai.

Nói ngắn gọn: không chỉ test “đi đúng nhánh”, mà còn test “dữ liệu đi đúng đường”.

Ví dụ Data Flow trong evaluate()

Lần theo biến watchlistHit

Biến này cho biết tài khoản nhận tiền có nằm trong watchlist không.

1) Gán

Repo tìm thấy tài khoản
⇒ `watchlistHit = true`

2) Dùng

`watchlistHit` được dùng để:

- thêm signal
- chọn PAUSE/WARN

3) Kiểm tra

Test nên assert:

- response có signal
- decision đúng
- event DB đúng

Điểm khác với Branch Coverage

BC chỉ hỏi “nhánh đã chạy chưa?” Data Flow hỏi thêm: “giá trị sau khi chạy nhánh đó có đi đúng tới response và DB không?”

JaCoCo — Đo độ phủ mã nguồn trong DigiShield

Cơ chế hoạt động của JaCoCo (Java Code Coverage)

Gắn Java Agent vào JVM khi chạy test; agent phân tích bytecode để ghi nhận chính xác từng chỉ lệnh (Instructions) và từng nhánh (Branches) đã được duyệt qua.

Lệnh chạy sinh báo cáo:

```
cd digishield
# Run tests and generate HTML report
./gradlew testCodeCoverageReport
# Open report in browser
open build/reports/jacoco/testCodeCoverageReport
/html/index.html
```

Cấu hình ngưỡng trong Gradle

File conventions.gradle.kts:

```
1 tasks.jacocoTestCoverageVerification {
2     violationRules {
3         rule {
4             limit {
5                 counter = "LINE"
6                 value = "COVEREDRATIO"
7                 minimum = "0.50".toBigDecimal()
8             }
9         }
10    }
11 }
```

Tự động đánh rớt build (FAIL) nếu code mới kéo độ phủ tụt dưới 50%!

Đọc báo cáo JaCoCo & Ý nghĩa thực tế

Màu sắc trên giao diện JaCoCo:

- **Kim cương Xanh lá:** Toàn bộ các nhánh rẽ tại dòng lệnh này đã được kiểm thử.
- **Kim cương Vàng:** Nhánh mới được phủ **một phần** (ví dụ biểu thức && mới chỉ test trường hợp False, chưa có True).
- **Kim cương Đỏ:** Nhánh hoàn toàn chưa được kiểm thử (0% coverage).

Độ phủ 100% $\neq$ Hết lỗi!

- **Thiếu logic:** Dev quên không viết code kiểm tra số dư âm $\rightarrow$ JaCoCo vẫn báo xanh 100% vì dòng code đó... chưa từng tồn tại!
- **Lỗi đặc tả:** Code chạy đúng thiết kế nhưng thiết kế bị sai nghiệp vụ.

Nguyên tắc vàng

Coverage là công cụ đo xem code đã được chạm tới chưa, không chứng minh được code đã xử lý hết mọi ngoại lệ.

Bài tập nộp sau buổi 3

Nội dung bài tập thực hành (LMS)

- Vẽ CFG, tính $V(G)$ cho `evaluate()` và `StubAiClient.classify()`.
- Áp dụng Loop Testing cho vòng lặp duyệt từ khóa trong `StubAiClient`.
- Thiết kế test case đạt 100% độ phủ nhánh (BC) cho `evaluate()`.
- Với Data Flow: chọn 1 biến trong `evaluate()` và mô tả biến đó được gán ở đâu, dùng ở đâu.

Quy cách nộp

Nộp file PDF bài làm lên LMS trước buổi học tiếp theo.

Chuẩn bị cho Buổi 4: Unit Testing với JUnit 5 & Mockito

Nội dung trọng tâm Buổi 4

- Khung kiểm thử đơn vị hiện đại: **JUnit 5** (Assertions, Lifecycle, Parameterized Tests).
- Kỹ thuật cô lập phụ thuộc: Test Doubles, Stubs vs Mocks.
- Framework **Mockito**: `@Mock`, `@InjectMocks`, `when( ).thenReturn( )`, `verify( )`.
- Viết unit test hoàn chỉnh cho Service layer của DigiShield đạt chuẩn JaCoCo.

Yêu cầu chuẩn bị trước ở nhà

- Chạy thử lệnh `./gradlew test` trên máy cá nhân để đảm bảo môi trường Java 25 hoạt động bình thường.
- Đọc trước tài liệu JUnit 5 User Guide (mục Assertions và `@ParameterizedTest`).

Tóm tắt buổi 3

Lý thuyết trọng tâm:

- **CFG**: Mô hình hóa luồng điều khiển thành đồ thị (Node, Edge, Decision node).
- **Độ phủ**: Câu lệnh (SC) $\subset$ Nhánh (BC) $\subset$ Đường đi (PC).
- **Cyclomatic** $V(G)$: đếm decision bằng công thức $V(G) = D + 1$.
- **Loop Testing**: Kiểm thử vòng lặp có cấu trúc tại các mốc biên $(0, 1, 2, m, n)$.
- **Data Flow**: Lăn theo biến từ nơi gán đến nơi dùng.

Thực hành trên DigiShield:

- Áp dụng CFG và $V(G)$ cho phần quyết định chính của `evaluate()` ($V(G) = 3$).
- Bộ 4 test case đạt 100% Độ phủ nhánh (BC) khớp với test thật trong repo.
- Lăn theo biến để kiểm tra dữ liệu đi đúng từ biến trung gian đến DB event và response.
- Đo độ phủ tự động bằng công cụ chuẩn công nghiệp **JaCoCo**.
- Hiểu rõ giới hạn: Hộp trắng cần kết hợp hộp đen để bao quát yêu cầu.

Tài liệu tham khảo

- [1] Slide bài giảng điện tử môn Kiểm thử Phần mềm, Bộ môn CNPM.
- [2] Paul Ammann & Jeff Offutt. *Introduction to Software Testing*, Cambridge University Press, 2008.
- [3] Glenford J. Myers. *The Art of Software Testing*, 2nd ed., Wiley, 2004.
- [4] ISTQB Foundation Level Syllabus, version 4.0, 2023. <https://istqb.org/>
- [5] Thomas J. McCabe. “A Complexity Measure.” *IEEE Trans. Software Eng.*, 1976.
- [6] JaCoCo Java Code Coverage Library, <https://www.jacoco.org/jacoco/>